

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 06 -

TESTES E CI PARA BIBLIOTECAS INTERNAS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	6
MERCADO, CASES E TENDÊNCIAS	20
O QUE VOCÊ VIU NESTA AULA?	26
REFERÊNCIAS.....	28

EMANIP

O QUE VEM POR AÍ?

Você já imaginou trabalhar em uma equipe onde cada desenvolvedor(a) contribui com pedaços de código para uma biblioteca interna importante, mas ninguém verifica se esses pedaços funcionam bem juntos até o último minuto? Seria como montar um carro às cegas: só na hora de dar a partida descobriríamos que parafuso sobrou ou qual peça está falhando – um receio constante de ver tudo “quebrar” sem aviso. No desenvolvimento de software esse medo é real: uma pequena mudança em uma função da biblioteca pode causar um efeito dominó de bugs em diversos sistemas que dependem dela.

Afinal, software é um ecossistema interconectado; sem testes automatizados, erros podem passar despercebidos e ressurgir no pior momento. Além disso, corrigir um bug depois que o sistema já está em produção pode custar muito mais tempo e dinheiro do que ter detectado o problema lá no início do desenvolvimento. É aqui que entram em cena os testes automatizados e a integração contínua (CI): eles atuam como uma rede de segurança que nos permite inovar e modificar o código da biblioteca sem perder a confiança de que nada essencial será quebrado.

Nesta aula, vamos explorar de forma estruturada como testes automatizados e Integração Contínua (CI) tornam nossas bibliotecas internas mais robustas e fáceis de manter. Começaremos entendendo por que testar é tão crucial – você verá como uma boa suíte de testes pode detectar erros precocemente, documentar o comportamento esperado e evitar regressões a cada mudança de código. Em seguida, vamos aprender a implementar testes unitários e de integração com frameworks populares em Python (como unittest e pytest), cobrindo casos de uso típicos e também aqueles extremos e “esquisitos” que não podemos negligenciar.

Finalmente, mergulharemos no mundo da Integração Contínua, descobrindo como configurar pipelines automáticos (com GitHub Actions, GitLab CI, Jenkins etc.) que executam nossos testes a cada nova atualização da biblioteca, garantindo qualidade de forma contínua e facilitando a colaboração de múltiplos desenvolvedores(as) sem dor de cabeça. Preparado(a)? Então vamos em frente – esse conhecimento vai elevar o nível dos seus projetos!

HANDS ON

As videoaulas mostram, passo a passo, como aplicar testes automatizados e CI em uma biblioteca interna Python. Em linguagem simples: você verá na prática como escrever alguns testes para uma função de exemplo e configurar um pipeline básico de Integração Contínua para rodar esses testes automaticamente a cada mudança no código. Escolhemos o Python pela sua popularidade e simplicidade – é uma das linguagens mais usadas tanto em engenharia de software quanto em ciência de dados e possui ótimas ferramentas de teste.

Começamos com uma função bem simples de uma biblioteca hipotética, por exemplo, `divide(a, b)`, que retorna o resultado da divisão de `a` por `b`. Nosso objetivo é garantir que essa função se comporte corretamente em vários cenários (incluindo casos extremos, como divisão por zero). Primeiro, veja a implementação dessa função interna da biblioteca:

```
def divide(a, b):  
    """Divide dois números, levantando um erro se o divisor for zero."""  
    if b == 0:  
        raise ValueError("Divisor não pode ser zero!")
```

Código-fonte 1 – Função `divide` da biblioteca interna, com tratamento básico de erro
Fonte: Elaborado pelo autor (2025)

Em seguida, criamos um arquivo de testes usando o framework `pytest` (poderíamos usar o `unittest` do Python padrão, mas o `pytest` torna o código de teste mais enxuto). Escrevemos testes unitários cobrindo diferentes cenários: um caso “normal” de divisão, um caso com número negativo e o caso extremo de divisão por zero (onde esperamos que a função lance um `ValueError`). Acompanhe:

```
# test_divide.py – Arquivo de testes usando Pytest  
from minha_biblioteca import divide  
import pytest  
  
def test_divisao_basica():  
    assert divide(8, 2) == 4  
  
def test_divisor_negativo():  
    assert divide(10, -2) == -5  
  
def test_divisao_por_zero():  
    with pytest.raises(ValueError):
```

```
divide(5, 0)
```

Código-fonte 2 – Exemplo de testes unitários em Python (Pytest) para a função divide
Fonte: Elaborado pelo autor (2025)

No início das videoaulas, executamos esses testes localmente com o comando `pytest` no terminal – todos devem passar se a função estiver correta. Em seguida, mostramos como integrar isso no GitHub Actions: adicionamos um arquivo de workflow (YAML) no repositório para que, a cada push ou pull request, o GitHub automaticamente instancie um ambiente, instale nossa biblioteca e rode `pytest`.

O vídeo exibe o pipeline em ação – subimos uma alteração intencionalmente errada (por exemplo, fazendo `return a // b` em vez de `a / b`) e vemos o pipeline falhar, acusando o teste quebrado. Depois, corrigimos o código e o pipeline passa a ficar “verde” de novo. Não deixe de assistir às videoaulas para ver todos esses passos na prática e entender como configurar isso nos seus próprios projetos. Ao final, você pode tentar reproduzir o exercício: pegar uma função da sua biblioteca, criar testes para ela e configurar uma integração contínua simples – essa experiência prática consolida de vez os conceitos aprendidos!

SAIBA MAIS

Vamos nos aprofundar nos aspectos técnicos de Testes Automatizados e Integração Contínua (CI), examinando porque essas práticas são tão vitais em projetos de software modernos e como aplicá-las de forma eficiente.

No desenvolvimento de software, testes automatizados servem como uma espécie de “seguro de qualidade”. Eles verificam continuamente se cada parte da biblioteca interna se comporta conforme o esperado e se novas mudanças não introduzem regressões (isto é, não quebram funcionalidades que já funcionavam). Lembre-se: um teste nada mais é do que um pequeno programa que checa outro programa. Quando você escreve `assert divide(8, 2) == 4`, está formalizando o que espera daquela função. Assim, se alguém (talvez você mesmo no futuro) acidentalmente alterar o código para `return a // b` – que em Python faz divisão inteira – o teste irá falhar, imediatamente sinalizando o erro antes que ele chegue a impactar usuários ou desenvolvedores(as) dependentes da biblioteca.

Um grande benefício dos testes automatizados é a detecção precoce de erros. Isso reduz drasticamente o custo de corrigir bugs: um defeito encontrado logo após ser introduzido é muito mais fácil (e barato) de resolver do que um problema descoberto semanas depois em produção, possivelmente após causar danos. Estudos clássicos de engenharia de software mostram que o custo de correção de um bug cresce exponencialmente quanto mais tarde no ciclo de desenvolvimento ele for encontrado. Em projetos academicamente controlados, já se viu que corrigir um erro em produção pode custar até 10x o esforço de tê-lo corrigido durante a implementação. Faz sentido – pense no retrabalho, nas possíveis perdas de dados ou insatisfação de usuários que são evitados ao pegar o bug lá atrás, assim que ele surge.

Além disso, uma suíte abrangente de testes automatizados funciona como uma documentação viva do sistema. Cada teste bem escrito diz: “Se você fornecer X, espero que a função retorne Y” ou “ao fazer tal operação, espero que aconteça tal coisa”. Isso ajuda novos(as) desenvolvedores(as) a entenderem o uso correto da biblioteca e as premissas das funcionalidades. Diferente de comentários em documentos, os testes não ficam desatualizados facilmente – se o comportamento

mudar, ou ajustamos os testes (se a mudança for intencional) ou os testes nos dirão que algo inesperado ocorreu.

Por isso, frequentemente se diz que testes bons aumentam a confiança no software. Quando você vê todos “verdinhos” passando, há uma sensação (justificada) de segurança para seguir adiante. Equipes maduras adotam até a prática de Test-Driven Development (TDD), em que desenvolvedores(as) escrevem os testes antes mesmo do código, definindo primeiro o comportamento esperado. Assim, o código nasce já “sabendo” o que tem que cumprir. Essa técnica pode ou não se adequar a todos, mas ilustra o valor que se dá aos testes como norteadores do desenvolvimento.

Resumindo: testamos para garantir qualidade, evitar surpresas desagradáveis e facilitar manutenção futura do código. Em contextos de bibliotecas internas – aquelas compartilhadas por vários projetos ou equipes – isso é ainda mais importante. Sem testes, seria arriscado alterar uma função usada em 5 sistemas diferentes; com uma boa cobertura de testes, conseguimos evoluir a biblioteca com a tranquilidade de que se algo for quebrado, descobriremos imediatamente (de preferência, antes de fazer o merge da alteração). E se tudo passar nos testes, temos um alto grau de confiança de que os sistemas permanecerão estáveis.

Tipos de testes: unitários vs. integração (e outros níveis)
Nem todos os testes são iguais. Adotamos diferentes níveis de teste para cobrir diferentes perspectivas do software. Os dois principais que nos interessam aqui são testes unitários e testes de integração, mas vale mencionar outros para situar onde cada um se encaixa:

- **Testes Unitários:** são o alicerce. Cada teste unitário verifica a menor unidade de código de forma isolada – geralmente uma função ou método. Por exemplo, testar a função divide em diversos cenários de entrada e saída. O objetivo é confirmar que, dado um conjunto específico de inputs, aquela unidade retorna o output correto ou lança o erro esperado. Unitários devem ser rápidos e independentes; se possível, não acessam banco de dados, rede ou sistema de arquivos – simulamos quaisquer dependências (usando mocks, por exemplo) para se concentrarem somente na lógica da unidade em teste. Quando bem feitos, eles garantem que “por dentro” cada peça do programa funciona direito. Você pode ter dezenas ou centenas de

testes unitários cobrindo todas as funções de uma biblioteca. Eles executam em segundos.

- **Testes de Integração:** são um nível acima. Em vez de focar em uma função isolada, eles avaliam a interação entre múltiplos módulos ou componentes. A ideia é verificar se as partes do sistema funcionam bem em conjunto. Por exemplo, imagine que sua biblioteca interna possua várias funções que, em sequência, realizam um cálculo complexo (pegar dados, filtrar, calcular resultado). Um teste de integração poderia chamar a funcionalidade completa (que internamente usa várias funções) e verificar o resultado final em um caso realista. Ou então testar a integração da sua biblioteca com um banco de dados ou API externa, se fizer parte do escopo.

Esses testes costumam ser mais demorados e mais complexos, pois podem envolver infraestrutura (simular um banco de dados de teste, arquivos etc.). Por isso, normalmente escrevemos menos testes de integração do que unitários – eles formam uma “camada” acima no famoso Test Pyramid (Pirâmide de Testes), conceito que recomenda muitos testes unitários na base, alguns testes de integração no meio e poucos testes end-to-end no topo. O racional é que os testes integrados cobrem cenários abrangentes, mas se algo falhar neles, pode ser mais difícil diagnosticar exatamente qual parte falhou; já os unitários apontam direto o módulo defeituoso. Portanto, ambos se complementam.

- **Testes de Sistema (End-to-End) e Testes de Aceitação:** são níveis ainda acima, envolvendo o sistema completo (por exemplo, usar a aplicação final como um todo, simulando um usuário). No contexto de uma biblioteca interna pura (sem interface gráfica ou API standalone), talvez não haja um “end-to-end” tradicional – esses aparecem mais em aplicações inteiras. Mas vale saber: testes end-to-end cobrem o fluxo completo em um ambiente real ou próximo do real. Já os de aceitação validam que os requisitos do cliente foram atendidos. Em projetos de produto final, esses níveis são cruciais; em bibliotecas internas, focamos mais nos unitários e integração, garantindo que a biblioteca fornece os componentes corretamente para que aplicações externas a usem.

- **Testes de Regressão:** esse termo não se refere a um nível distinto, mas ao objetivo de um conjunto de testes. Uma suite de regressão é basicamente um conjunto amplo de testes (de vários níveis) que você roda após cada mudança para confirmar que nada foi quebrado. Quando dizemos “rodar os testes de regressão”, significa executar todos (ou um subconjunto crítico) dos testes já existentes para verificar se há alguma novidade falhando onde antes tudo passava. A prática de CI, como veremos, automatiza justamente isso: sempre que integramos algo novo, rodamos todos os testes automaticamente. Se algum falhar inesperadamente, detectamos uma regressão imediatamente. Em suma, pelos testes unitários e de integração bem escritos, prevenimos regressões no código – daí a importância de mantê-los sempre atualizados e expandi-los conforme novas funcionalidades são adicionadas.

Resumindo em termos práticos: testes unitários cobrem as peças individuais e testes de integração cobrem o funcionamento do conjunto. Ambos são necessários. Imagine uma calculadora científica – você testa separadamente as funções de soma, subtração, multiplicação (unitários) e depois testa uma expressão completa envolvendo várias operações (integração). Se todos unitários passam mas um teste integrado falha, talvez as funções individualmente funcionem, porém há um problema no encadeamento (quem sabe uma variável global mal utilizada, ou uma formatação incorreta ao combinar resultados). Por outro lado, se o teste integrado falha e não havia unitários, fica difícil saber qual passo interno deu problema. Por isso investimos na base (unitários) primeiro, mas sem abrir mão de testar as interações críticas entre componentes.

Implementando testes na prática: ferramentas e boas práticas

No ecossistema Python, contamos com excelentes ferramentas para escrever e organizar testes. Já citamos as duas mais relevantes: o módulo nativo unittest e o framework pytest (terceiro, mas amplamente adotado). Vamos compará-los brevemente:

CRITÉRIO	UNITTEST (STDLIB)	PYTEST (3RD PARTY)
Sintaxe	Verbosa, baseada em classes e métodos especiais (TestCase, setUp etc.).	Concisa e legível, usa funções e simples assertivas Python (assert).
Descoberta de testes	Requer definir classes que herdam TestCase e métodos começando com test_. Uso de unittest.main() ou similar para rodar.	Automática: qualquer função ou método cujo nome comece com test_ em arquivos nomeados test_*.py é reconhecido. Basta rodar pytest na pasta e ele encontra tudo.
Setup	Usa métodos como setUp/tearDown em classes para preparar estado antes/depois de cada teste. Suporte a fixtures agrupados, mas menos flexível.	Possui um poderoso sistema de fixtures via decoradores, permitindo reutilização de código de preparação e escopos (por função, módulo, sessão etc.). Facilita muito montar cenários complexos de teste.
Assertões	Usa métodos de asserção do TestCase (ex.: self.assertEqual(x, y), assertTrue(cond) etc.). Output de erro às vezes menos intuitivo (precisa interpretar mensagem do framework).	Usa diretamente o assert do Python, mas introspectivo: em caso de falha, o pytest mostra os valores concretos de cada lado da comparação, facilitando debugar. Menos código repetitivo (sem self. e métodos).
Extensibilidade	Suportado em IDEs e muito estável (faz parte da stdlib). Plugins existem (p. ex. unittest.mock para mocks), mas estrutura geral é fixa.	Altamente extensível: enorme ecossistema de plugins (coverage, pytest-django, pytest-xdist para paralelizar etc.). Permite parametrização de testes facilmente (rodar o mesmo teste com múltiplas entradas) e outras mágicas.

Curva de aprendizado	Baixa, pois segue estilo parecido com JUnit de Java (familiar pra muitos). Porém requer escrever mais código boilerplate para casos simples.	Muito simples para começar (escreva funções e use assert), porém oferece recursos avançados (fixtures, markers) que levam algum tempo para dominar completamente. Documentação rica e comunidade ativa ajudam.
----------------------	--	--

Tabela 1 – Comparação entre frameworks de teste em Python: unittest vs pytest
Fonte: Elaborado pelo autor (2025)

Em resumo, unittest é ótimo por vir embutido no Python (não precisa instalar nada) e seguir uma abordagem tradicional xUnit (similar a JUnit, NUnit etc.), garantindo organização e resultados padronizados. Já o pytest conquistou os desenvolvedores e desenvolvedoras Python pela praticidade – remove muita verbosidade e oferece recursos de alto nível para tornar os testes mais expressivos e abrangentes. Hoje, muitos projetos preferem pytest pela velocidade de escrever testes e pela facilidade de integração com outras ferramentas, inclusive CI (ele gera saídas de teste padronizadas que podem ser capturadas por Jenkins, GitHub Actions e afins).

Vale notar que não é “um ou outro”, podemos usar ambos no mesmo projeto se quisermos. O pytest inclusive roda casos de teste escritos no estilo unittest sem problemas – então equipes em transição podem adotar pytest gradualmente sem descartar o legado (pytest detecta as classes TestCase do unittest e as executa também).

Independentemente da ferramenta, existem boas práticas na escrita de testes:

- **Nomenclatura clara:** nomeie seus testes de forma descritiva. Por exemplo, `test_divisao_por_zero_deve_lancar_erro()` já deixa claro o que está sendo validado. Isso ajuda quando um teste falha – pelo nome já entendemos o requisito não atendido.
- **Isolamento:** cada teste deve ser independente dos outros. Não dependa de variáveis globais modificadas por outro teste, nem da execução sequencial. Se a ordem de execução alterar o resultado, seus testes não são confiáveis (pode mascarar bug ou dar falsos negativos). Ferramentas como pytest executam testes em ordem não determinística justamente para incentivar

isolamento. Use fixtures para setUp/tearDown comuns, mas evite compartilhar estado mutável entre testes.

- **Cobrir casos normais e extremos:** para cada funcionalidade, pense nos casos de uso típicos (inputs válidos comuns) e também nos bordos e errados. Ex.: na função divide, casos típicos: divisão de inteiros positivos, de números negativos, talvez floats; casos de borda: dividir por 1, dividir 0 por algo; caso de erro: divisor zero (espera exceção). Uma estratégia é usar análise de partição e valores de fronteira (técnicas de teste de caixa-preta da engenharia de software) – dividir o domínio de entrada em classes de equivalência e testar pelo menos um valor de cada classe e valores limítrofes entre elas. Assim pegamos condições de contorno que frequentemente causam problemas (um clássico é off-by-one em loops, que testamos com tamanho 0, 1, e vários).
- **Teste primeiro funcionalidades “core”:** priorize escrever testes para as partes mais críticas da biblioteca – aquelas sem as quais nada funciona ou que fazem cálculos complexos. Por exemplo, se sua biblioteca interna é de cálculo financeiro, tenha muitos testes assegurando a exatidão desses cálculos. Detalhes cosméticos ou triviais (como uma função que apenas formata uma string) podem esperar, embora cobrir tudo seja ideal quando houver tempo.
- **Mantenha testes rápidos:** se os testes demorarem demais, os desenvolvedores e desenvolvedoras podem hesitar em rodá-los com frequência. Para bibliotecas internas, geralmente não há interface gráfica ou necessidade de esperar muitos segundos – então procure escrever testes que rodam em milissegundos. Se um teste de integração precisar acessar um recurso lento (por ex., fazer download de um arquivo grande), pense em estratégias de como simular esse recurso ou usar um subconjunto menor. Em CI, isso conta muito: builds longos podem atrapalhar a agilidade. A recomendação do próprio Martin Fowler para CI é “Keep the build fast” (manter o build rápido), preferindo uma suíte de testes rápida que roda a cada commit, e talvez testes mais pesados rodando apenas periodicamente (como diários).

- **Verifique também comportamentos negativos:** não só o “caminho feliz” (happy path) onde tudo dá certo, mas também situações de erro esperadas. Se sua função deve lançar exceção em certo contexto, escreva um teste confirmando isso (como fizemos com o `pytest.raises` no nosso exemplo). Isso garante que você não só codificou o comportamento correto como também não removeu acidentalmente a validação no futuro. É comum, durante refatorações, alguém tirar uma checagem de erro julgando desnecessária – um teste quebra na hora e avisa: “opa, agora passar 0 não está mais levantando erro como deveria”.
- **Evite falsos positivos/flaky tests:** um teste “flaky” é aquele que às vezes passa, às vezes falha, sem mudança de código – frequentemente devido a dependências de ambiente ou temporizações. Esses testes minam a confiança, porque não se sabe se falhou por bug real ou instabilidade do teste. Em bibliotecas puras, flakiness é raro (geralmente acontece mais em testes de interface ou integração com rede). Mas se tiver testes dependentes de tempo (ex.: esperando 1 segundo), seja cuidadoso com margens. Idealmente, testes devem ser determinísticos.
- **Use mocks (simulações) apropriadamente:** em testes unitários, para isolar a unidade, usamos mocks para substituir dependências externas. Ex.: se sua função consulta uma API, em vez de fazer chamada real (que seria lenta e instável), você “finge” a resposta. O Python fornece o módulo `unittest.mock` para isso (também integrável ao `pytest`). Porém, use mocks com critério – muitos mocks podem transformar o teste em uma validação de implementação interna em vez do comportamento. Uma regra útil: não mockar o que você não possui. Ou seja, simule apenas sistemas externos ou camadas que fogem ao seu controle (como chamadas HTTP, banco de dados), mas não fique simulando funções internas da própria unidade sob teste; se fizer isso, você talvez teste se o código chama funções X e Y, mas não se o resultado final está correto. Em integração, preferimos não mockar nada – usar componentes reais juntos.
- **Mantenha a suíte de testes atualizada:** sempre que um bug for descoberto na biblioteca, escreva primeiro um teste que reproduza o problema – assim você “congela” aquele cenário para garantir que não volte a ocorrer (teste

de regressão). Após consertar o bug, esse novo teste deve passar e permanecer na suíte para futuras execuções. Com o tempo, a suíte de testes se torna mais rica e cobre os pontos frágeis já conhecidos. Também adapte ou aumente os testes conforme novas funcionalidades forem adicionadas. Testes defasados (que não refletem mais a especificação) devem ser ou corrigidos ou removidos – ter testes “quebrando” sem motivo válido apenas faz o time ignorar os relatórios de teste, o que é péssimo.

Poderíamos dedicar um curso inteiro só para técnicas de teste de software, mas esse panorama já nos equipa para escrever testes decentes para nossa biblioteca interna. Em projetos de ciência de dados, essas práticas também se aplicam! Por exemplo, se sua biblioteca interna contém funções de limpeza de dados ou cálculo de estatísticas, escreva testes garantindo que “dados sujos X resultam em dados limpos Y”. Se há uma função que treina um modelo, você pode testar em um dataset reduzido e verificar se as métricas atingem um patamar esperado (ex.: “teste que, com dataset de brincadeira, a acurácia é pelo menos 90%”). Até algoritmos de aprendizado podem (e deveriam) ter testes – talvez não do resultado exato (que pode variar com random seeds), mas de comportamentos: “o modelo converge em menos de N iterações”, ou “tal função de custo decresce quando fornecemos exemplos bem classificados”.

São formas de assegurar que refatorações no código de ML não piorem a performance. Ferramentas como Hypothesis (testes baseados em propriedade) podem gerar automaticamente múltiplos inputs aleatórios para testar invariantes – uma abordagem interessante para funções matemáticas ou lógicas complexas. E lembre-se: assista às videoaulas novamente se tiver dúvidas de como estruturar na prática um arquivo de testes, rodá-los ou usar asserts e mocks – rever a implementação ajuda a fixar as melhores práticas.

Agora que sabemos escrever testes, como garantir que eles sejam executados religiosamente a cada modificação do código? É aí que entra a Integração Contínua (Continuous Integration). O termo pode soar um jargão, mas vamos destrinchá-lo. Integração Contínua é uma prática de engenharia que envolve integrar (combinar) continuamente as alterações de código de todos os desenvolvedores e desenvolvedoras em um repositório central e verificar a consistência dessas integrações através de builds e testes automáticos.

Em outras palavras, sempre que alguém faz um commit ou abre um pull request na biblioteca, um servidor de CI pega aquele novo código, constrói o projeto (se aplicável) e roda todos os testes, reportando sucesso ou falha. Assim, problemas de integração são detectados o mais cedo possível. Martin Fowler, um dos gurus de metodologias ágeis, define assim: “Continuous Integration is a practice where each team member merges their work into the main codebase at least daily; each integration is verified by an automated build (including tests) to detect errors quickly”. O resultado? Redução de riscos e de esforços de integração manual, além de um código que está quase sempre em estado pronto para release, já que a qualquer momento (com testes passando) ele teoricamente está estável para produção.

Imagine voltar alguns anos no tempo, antes do CI se popularizar: equipes trabalhavam em ramas separadas por semanas e só integravam tudo no final de um ciclo – o famigerado “big bang merge”. Os testes eram rodados manualmente por QAs eventualmente. Integração era um pesadelo demorado, cheio de conflitos de código e bugs inesperados. Com CI, integramos cedo e frequentemente, tornando integração algo rotineiro e sem sobressaltos. A cada pequena junção de código nova, validamos automaticamente. Se algo quebra, sabemos exatamente qual mudança causou (porque foi a integração imediata anterior) e já corrigimos. Tanto que uma das regras do CI é: quebrou a build, pare tudo e conserte imediatamente antes de qualquer outra coisa. Isso evita que erros se acumulem.

Para implementar CI, normalmente usamos pipelines automatizados em servidores ou serviços especializados. Hoje existem diversas opções (falaremos de algumas), mas todas seguem a mesma ideia: observar o repositório de código e, em determinados gatilhos (por exemplo, “novo commit na branch main” ou “abertura de PR”), disparar jobs de build/teste. No caso da nossa biblioteca Python, o pipeline pode ser simples:

1. Instalar a versão atual do Python no ambiente de CI.
2. Baixar o código da biblioteca (checkout do repositório).
3. Instalar as dependências (se há outras libs requeridas).
4. Rodar os testes automatizados (geralmente executando `pytest` ou `python -m unittest`).

5. (Opcional) Calcular métricas como cobertura de testes ou executar linters etc.
6. Reportar o resultado (sucesso ou falha), possivelmente mandando notificações ou exibindo em um dashboard.

Tudo isso sem intervenção humana, dentro de alguns minutos. Se todos os testes passarem, a integração é “verde” – o código integrado está ok. Se algum falhar, a integração fica “vermelha” – deve-se corrigir antes de prosseguir. Muitas equipes configuram seu sistema de controle de versão para exigir CI verde antes de permitir merge em branches protegidas. Por exemplo, no GitHub, você pode proteger a branch principal para só aceitar pull request depois que o GitHub Actions confirmar que todos os testes passaram naquele PR. Isso força “disciplina de qualidade” automática.

Uma dúvida comum: e se o projeto for muito grande? Resposta: boas práticas de CI continuam válidas – nesses casos fazemos otimizações. Por exemplo, suponha que sua biblioteca tenha milhares de testes, levando 30 minutos para rodar tudo. Ainda assim, integrar frequentemente é benéfico; mas podemos paralelizar a execução (rodar testes em múltiplos agentes em paralelo) ou usar técnicas de regression test selection para rodar prioritariamente testes relacionados às partes de código alteradas. Ferramentas de CI modernas (e.g., GitLab CI, Azure DevOps) permitem dividir a suíte de testes em lotes e executar simultaneamente, reduzindo o tempo total.

Outro truque é manter um pool de máquinas ou containers prontos para uso, assim não se perde tempo provisionando ambiente a cada execução. A regra de ouro é manter o feedback rápido: idealmente, o(a) desenvolvedor(a) abre um pull request e em poucos minutos sabe se algo quebrou. Se testes estão demorando horas, perde-se a agilidade (nesse caso, investe-se em otimização ou em separar pipelines – por exemplo, smoke tests rápidos a cada commit e uma bateria completa rodando toda noite).

Os principais serviços/ferramentas de CI empregados hoje em dia são muito fáceis de usar. No passado, configurar um pipeline de CI muitas vezes exigia configurar seu próprio servidor e cuidar de toda infraestrutura. Hoje temos alternativas em nuvem muito fáceis de usar, integradas aos repositórios como Jenkins, GitHub Actions e GitLab CI/CD. A escolha costuma depender do ecossistema em que você já está: equipes usando GitHub tendem a optar pelo Actions pela conveniência, quem já

está no GitLab usa o CI interno que é excelente, empresas com compliance estrito ou necessidades muito customizadas às vezes ficam com Jenkins por controle total.

Notavelmente, a automação de testes via CI se tornou padrão da indústria – um relatório de 2024 indicou que 83% dos desenvolvedores e desenvolvedoras já estão envolvidos em práticas de DevOps como CI/CD. E não é só adotar a ferramenta, é colher resultados: o mesmo relatório mostra que o uso de ferramentas de CI/CD se correlaciona a melhor desempenho de entrega de software (medido pelas métricas DORA – frequência de deploys, tempo de restauração etc.). Ou seja, times com CI funcionando tendem a entregar features mais rápido e com menos bugs em produção.

Empresas líderes como Google e Facebook integraram essa cultura cedo – no Google, como mencionamos, nada menos que 120 milhões de testes são executados diariamente para garantir que cada mudança no imenso repositório monolítico não degrade nada. No Facebook, existiam histórias de engenheiros habilitando 50 mil máquinas de teste em paralelo para rodar a suíte inteira em pouco tempo. Esse nível de investimento em testes automatizados e CI é o que permite releases diários ou múltiplos deploys por dia sem esfarelar a qualidade.

No contexto de bibliotecas internas, a Integração Contínua atua como guardiã da qualidade para todos os consumidores dessa biblioteca. Imagine que sua empresa tenha uma biblioteca interna de funções estatísticas usada por todos os cientistas de dados. Com CI, cada vez que alguém da equipe de plataforma modifica essa biblioteca, todos os testes rodam; se algo falhar, a equipe é notificada e não libera uma versão defeituosa. Isso significa que as equipes de ciência de dados que usam a lib não serão surpreendidas por uma atualização cheia de bugs – idealmente, elas nem verão bug nenhum, porque ele já teria sido pego no CI e consertado antes de promover a mudança.

Além disso, integrar com frequência previne o “drift” de branches: desenvolvedores(as) não ficam meses isolados(as) com suas alterações – pelo contrário, enviam pequenas parcelas contínuas que são imediatamente verificadas e mescladas. Isso incentiva colaboração e melhoria incremental. Se múltiplos devs podem contribuir em uma mesma biblioteca, o CI se torna o ponto de encontro que garante que as contribuições de todos joguem bem juntas. Lembra do caos hipotético de muitos códigos sem teste se juntando? Com CI, viramos esse jogo: as integrações acontecem toda hora, mas de forma controlada e monitorada.

Um pipeline de CI pode também executar outros tipos de verificações além dos testes de unidade/integrados tradicionais. Por exemplo:

- **Linters e formatadores:** verificar estilo de código (PEP8) ou problemas estáticos (pylint, flake8) – muitas orgs fazem o CI falhar caso o código não atenda a certos padrões, forçando qualidade consistente.
- **Análise de cobertura de testes:** medir qual percentual do código da biblioteca é exercitado pelos testes. Se cair abaixo de um limiar (digamos, 80%), talvez marcar build como aviso ou mesmo falha (alguns projetos exigem, por política, cobertura mínima). Mas cuidado: perseguir 100% de cobertura às cegas não garante ausência de bugs, pode levar a testes artificiais; o indicador deve ser usado com bom senso. Ainda assim, ter cobertura alta geralmente indica boa abrangência de testes, o que é positivo. Ferramentas como coverage.py geram relatório; integradas no CI, podem até comentar em um PR quão coberto está o novo código.
- **Testes de desempenho (microbenchmarks):** embora menos comum em CI de cada commit (porque podem demorar), algumas pipelines incluem execução de benchmarks padronizados para detectar regressões de performance. Imagine uma biblioteca interna de algoritmos – seria útil saber se um commit novo tornou a função 30% mais lenta. Existem plugins pytest (pytest-benchmark) que ajudam; no CI podemos guardar histórico e alertar se degradou além de um limiar.
- **Verificação de segurança:** como apontado na tabela, o GitLab CI facilita rodar scanners de vulnerabilidades automaticamente. Em Python, podemos integrar tools como bandit (que encontra possíveis falhas de segurança no código) e até rodar checagem de dependências (ex.: alertar se alguma biblioteca usada tem CVE conhecido). O DevSecOps incentiva “shift left” da segurança – em vez de testar segurança só no final, já incluir no CI. Para bibliotecas internas, isso significa garantir que nenhuma mudança introduziu, por exemplo, uso de método inseguro de criptografia ou exposição de segredo sem querer (já houve casos reais de commit expondo senha ou chave de API e pipelines agora conseguem detectar certos padrões e barrar o commit, protegendo a empresa).

- **Build de pacotes e release automatizado:** indo além, algumas pipelines de CI já fazem a biblioteca interna ser empacotada e publicada (em um repositório privado, como Azure Artifacts, Artifactory ou PyPI interno) quando uma mudança é aprovada. Isso entra em Continuous Delivery – se tudo passou, a atualização já pode ficar disponível para uso. Por exemplo, incrementar versão e disponibilizar o pacote .whl / .tar.gz da biblioteca. Equipes avançadas têm fluxos assim: commit -> testes -> pacote gerado -> deploy (no caso de libs, “deploy” é publicar para quem usa). Assim, os consumidores sempre podem pegar a versão mais recente validada.

Falando em melhores práticas e tendências em CI, uma prática muito útil é configurar gatilhos de pipeline diferenciados. Por exemplo, rodar todos os testes unitários a cada commit (rápido) e rodar testes de integração mais pesados somente em merges para a main ou uma vez por noite. Ou talvez rodar um pipeline curto nos commits (lint + unit) e, em paralelo, um pipeline completo (unit + integração + outros checks). Isso maximiza o feedback rápido sem abrir mão da cobertura completa eventual. Ferramentas de CI permitem definir múltiplos fluxos com condições.

Outra tendência forte é o uso de contêineres/Docker nos pipelines, pois padroniza ambientes, garantindo que “na minha máquina” = “na máquina do CI”. Em nosso caso Python, podemos fixar a versão do interpretador e libs, evitando diferenças. Também fica fácil testar a biblioteca em múltiplas versões do Python – pipelines podem parametrizar o job em um matrix (ex.: testar com Python 3.8, 3.9, 3.10 e 3.11), assim temos certeza de compatibilidade com todos (importante para bibliotecas disseminadas internamente). Isso seria custoso manualmente, mas o CI faz em paralelo, de modo transparente.

E no mercado, o que vemos? Ferramentas de CI evoluíram tanto que hoje se fala mais em CI/CD em conjunto – ou seja, além de integrar e testar continuamente, muitas empresas também implantam continuamente (Continuous Deployment) as novas versões aprovadas. Em bibliotecas internas, “implantar” pode ser liberar automaticamente a nova versão para equipes usarem, ou atualizar um serviço que depende da nova função. A tendência geral é encurtar cada vez mais o ciclo entre escrever código e disponibilizá-lo com qualidade. Nesse cenário, testes automatizados robustos são imprescindíveis – eles são o artilheiro principal da etapa de verificação no pipeline de DevOps.

MERCADO, CASES E TENDÊNCIAS

A adoção de testes automatizados e integração contínua transformou profundamente a indústria de software nas últimas duas décadas. Hoje, é praticamente impensável um projeto de grande porte sem essas práticas – elas são o alicerce do movimento DevOps e de metodologias ágeis de entrega contínua. Vamos analisar alguns casos reais, tendências de mercado e até histórias de fracasso e sucesso que giram em torno de testes e CI:

De diferencial a obrigatório: no passado, entregar software com um bom conjunto de testes era um “bônus”. Atualmente, é esperado. Grandes empresas de tecnologia forjaram essa cultura – e influenciaram todo o mercado. Por exemplo, a Microsoft adotou internamente o mantra “testar tudo” após enfrentar, nos anos 2000, diversos bugs críticos em produtos; hoje, produtos como Windows e Office possuem milhões de testes automatizados rodando antes de cada release.

Uma pesquisa do Continuous Delivery Foundation de 2024 mostrou que 83% dos desenvolvedores e desenvolvedoras já participam de atividades DevOps como CI/CD. Ou seja, a maioria das equipes já incorporou pipelines automáticos ao fluxo de trabalho. Esse amadurecimento veio porque funciona: os mesmos estudos associam o uso extensivo de CI/CD a menores taxas de falha em deploys e recuperações mais rápidas quando problemas ocorrem. Em termos simples, times “elite” em entrega de software são justamente os que abraçaram testes automatizados e CI de forma séria. Por isso, startups e empresas menores também correm para implementar essas práticas, pois se tornaram sinônimo de profissionalismo e qualidade no mercado.

Cases de sucesso (e escala): já comentamos do Google – um caso impressionante. No artigo “The State of Continuous Integration Testing @ Google”, John Micco revela números do ecossistema interno: são executados 150 milhões de testes por dia, com 99% de taxa de sucesso (ou seja, a maioria das execuções passa, os 1% que falham são revertidos ou corrigidos rapidamente). Eles conseguiram fazer da integração contínua uma verdadeira linha de montagem automatizada de software. Esse investimento maciço permite que o Google lance atualizações para seus serviços diariamente (Gmail, Search etc.) sem interrupções – qualquer problema foi pego em fases anteriores.

Outro caso famoso é o do Facebook (Meta). A empresa implementou há anos um sistema em que cada diff (mudança de código) gatilha um conjunto enorme de testes e análises estáticas; só é possível fazer merge no branch principal se tudo passar e pelo menos um revisor humano aprovar. O Facebook chegou a reportar que faz despliegues 2 vezes por dia da sua aplicação web, sustentados por essa retaguarda de testes robusta e CI. Empresas de comércio eletrônico como a Amazon adotam filosofia similar – “you build it, you run it” – ou seja, cada time de desenvolvimento é responsável por testar e monitorar seu pedaço e usa pipelines automáticos para entregar features em produção continuamente (a Amazon é conhecida por fazer deploy centenas de vezes por dia em diferentes serviços!).

Esses casos inspiram o resto do mercado: hoje até setores tradicionais (bancos, indústria automotiva) correm atrás de melhorar suas pipelines de teste e integração, visando igualar esse nível de agilidade e qualidade.

Lições de falhas históricas: infelizmente, muitas organizações só perceberam o valor de testes/CI após viverem desastres causados por falta dessas práticas. Há diversos casos documentados. Um emblemático: o foguete europeu Ariane 5 explodiu em 1996 apenas 37 segundos após o lançamento, causando uma perda de US\$ 370 milhões. A causa foi um bug de software – um erro de conversão numérica (overflow) que não foi detectado porque um módulo reutilizado da Ariane 4 não havia sido testado para os valores maiores da Ariane 5. Investigações concluíram que testes intensivos de simulação poderiam ter pego o erro antes do voo.

Da mesma forma, em 1999, a NASA perdeu a sonda Mars Climate Orbiter por uma falha: uma equipe usava unidades inglesas (pés/libras) e outra métricas (metros/Newtons), resultando em cálculo de trajetória errado – a nave se desintegrou ao entrar na atmosfera marciana, um prejuízo de US\$ 327 milhões. Novamente, concluiu-se que faltou integração e teste entre os módulos das equipes: se tivessem realizado um teste de integração que validasse as unidades de medida, o erro teria sido captado.

Exemplos trágicos abundam: há o caso do Therac-25 (máquina de radioterapia nos anos 1980) em que bugs de software causaram overdose de radiação, levando a óbitos – ali ficou evidente a ausência de testes robustos de integração entre o software e os controles de hardware de segurança. Também temos exemplos de perdas financeiras: a bolsa de Londres interrompeu operações por 7 horas em 2012 por conta

de uma atualização sem testes adequados; ou o famigerado caso da Knight Capital em 2012, quando uma corretora de Wall Street implantou um código novo sem testar corretamente em todos os servidores – um módulo legado reativado erroneamente começou a executar ordens loucamente e, em 45 minutos, a empresa perdeu \$440 milhões, praticamente quebrando-a. A Knight Capital teve que ser vendida às pressas depois desse evento.

Análises mostraram que diversos erros foram cometidos: não havia uma suíte abrangente de testes de regressão para pegar o “efeito Power Peg” e não havia um procedimento seguro de rollback (o rollback manual feito às pressas na verdade replicou o bug em todos os servidores). Esses casos viraram estudos de engenharia de software e o ponto comum é: falhas de teste e integração podem causar prejuízos enormes ou até perdas de vidas. Eles serviram de alerta para muitas organizações investirem pesado em qualidade preventiva.

DevOps e culture shift: a popularização de testes e CI caminha de mãos dadas com a cultura DevOps. Hoje não há silos estanques de “desenvolvimento joga código para QA testar manualmente”. Times modernos adotam o conceito de “shift left” – testar e integrar o quanto antes, idealmente pelo próprio indivíduo desenvolvedor enquanto codifica, usando automação. O papel do QA evoluiu de executar testes manuais repetitivos para criar ferramentas e cenários automatizados. Nas grandes empresas, muitas vezes há engenheiros de software em teste (SET/QA Engineers) focados em ampliar a cobertura de testes, criar frameworks internos e monitorar pipelines. Por exemplo, no Google existe o papel de SETI (Software Engineer in Test Infrastructure).

Com essa mentalidade, surge a prática de CI contínuo até em estações locais: desenvolvedores(as) executam testes locais antes de cometer código para já encontrar erros (às vezes integrado no próprio editor/IDE rodando a cada salva). A CI no servidor então quase nunca falha, porque todo mundo já validou antes – ela vira apenas um cheque final e uma garantia de padronização. Essa mudança cultural de “todos são responsáveis pela qualidade” fez com que as linhas entre dev e ops ficassem borradas – desenvolvedores(as) monitoram pipelines, ops às vezes escrevem testes infra. O resultado é maior responsabilidade e colaboração.

Ferramentas integradas e tendências técnicas: no que tange a ferramentas, vimos comparações. Hoje há uma tendência de consolidação: GitHub Actions e GitLab

CI estão crescendo muito e tomando espaço de soluções legadas como Jenkins, especialmente entre projetos novos e open-source. O fato de serem as-a-service (sem precisar instalar servidor) e fortemente integrados com onde o código está traz conveniência imensa. Por outro lado, Jenkins ainda é onipresente em corporações, até pela longa história – muitos times têm pipelines complexos rodando nele e seria custoso migrar.

No âmbito dos testes, o pytest virou praticamente o padrão de fato em Python pela sua flexibilidade; frameworks de BDD como behave existem, mas são menos usados em desenvolvimento interno (BDD aparece mais para testes de aceitação com stakeholders não técnicos). Ferramentas de análise estática e linters também se integram ao pipeline (ex.: SonarQube para qualidade de código global). Uma tendência muito forte agora é incorporar testes de segurança e conformidade no CI: termos como “DevSecOps” significam que pipeline não testa só funcionalidade, mas também verifica segurança e compliance (por exemplo, garantindo que uma mudança de código não violou uma regra regulatória).

Automação de testes auxiliada por IA: uma novidade “quentíssima” (e polêmica) é o uso de Inteligência Artificial para gerar testes automaticamente. Ferramentas emergentes – por exemplo, o Qodo Cover (inspirado por um paper da Meta/Facebook) – prometem usar modelos de linguagem para analisar seu código e criar casos de teste unitário adicionais, aumentando a cobertura. A ideia é interessante: a IA escreve testes que talvez o(a) desenvolvedor(a) não pensou, potencialmente achando cenários de canto. Contudo, ainda é incipiente; relatos mostram que para códigos complexos a IA tropeça devido à falta de contexto completo.

Empresas como Diffblue e outras startups lançaram produtos de “AI for Tests” focados em Java e já há plugins para Python também. A tendência aponta que, no futuro, talvez parte do trabalho braçal de escrever testes possa ser acelerado por IA – o engenheiro vira um curador que valida/ajusta os testes gerados. Mas é consenso que isso não substitui um design cuidadoso de testes: a IA pode montar cenários, mas entender requisitos e escolher equivalências requer entendimento humano. Ainda assim, fique de olho: possivelmente, uma tendência futura será pipelines de CI integrando bots de IA para gerar novos testes a cada PR e até refatorar códigos de teste. Isso casaria bem com outra ideia emergente: Self-healing tests – suítes de teste

que se ajustam automaticamente a pequenas mudanças de comportamento esperado (claro, com validação humana depois).

QA além do código: vale citar que a automação de testes se expande até fora do código-fonte. Por exemplo, há bibliotecas internas em áreas operacionais (infraestrutura como código, scripts de deployment) que também podem (e devem) ser testadas. Ferramentas de teste de infra (como Terratest para Terraform) surgiram para aplicar os mesmos princípios de testes unitários/integração em código de infraestrutura. Ou seja, a mentalidade de “tudo como código, tudo testável” está ganhando terreno. Empresas de ponta aplicam teste contínuo até em pipelines de dados (DataOps) – validando integridade de dados conforme fluxos ETL rodam, por exemplo.

Enquanto isso, observabilidade e testing em produção são tópicos atuais: técnicas como chaos engineering (injetar falhas controladas em produção para ver se o sistema aguenta) e monitorações avançadas complementam os testes tradicionais, cobrindo o que não se consegue simular completamente. Isso reforça que testes e CI não são a solução de tudo – são peças de uma abordagem mais ampla de qualidade contínua. Mesmo com 100% de testes passando, ainda monitoramos o software rodando de verdade, pois sempre há casos inesperados. Mas sem os testes, chegar ao ponto de pôr em produção já seria arriscado demais.

Em termos de mercado de trabalho, habilidades em automação de testes e CI/CD são cada vez mais demandadas para indivíduos desenvolvedores e engenheiros de dados. Olhar vagas atuais mostra requisitos como “experiência com pipelines CI/CD (Jenkins, GitLab ou GitHub Actions)” e “conhecimento de frameworks de teste”. Certificações DevOps e treinamentos focados em ferramentas específicas estão em alta. Comunidades open-source colaboram intensamente: por exemplo, quase todo projeto importante no GitHub possui ações integradas de build. Isso encoraja até contribuintes casuais a rodar testes localmente antes de enviar código público, espalhando a cultura de qualidade.

Resumindo as tendências: mais automação, mais cedo, mais integrada e mais inteligente. Testes e CI caminham para se tornar invisíveis e onipresentes – algo que “apenas acontece” nos bastidores de cada commit. O(a) desenvolvedor(a) do futuro talvez só perceba a presença de IA sugerindo testes ou o pipeline enviando um relatório no chat informando “Sua mudança piorou o tempo de resposta em 5%,”

verifique se é esperado”. O papel humano continuará fundamental para pensar cenários e interpretar resultados, mas muito do trabalho repetitivo e de coordenação será absorvido por ferramentas. E isso é uma ótima notícia: significa que teremos menos “bugões” e falhas catastrófica, e poderemos gastar mais tempo inovando do que apagando incêndios.

EMENDAS

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, abordamos de forma abrangente testes e integração contínua para bibliotecas internas. Entendemos a importância dos testes automatizados no contexto de bibliotecas internas, vendo como eles garantem que cada funcionalidade individual funcione como esperado e previnem regressões a cada mudança. Discutimos que testar desde cedo economiza tempo/custo e aumenta a confiança no código.

Aprendemos a implementar testes unitários e de integração usando frameworks Python (mostramos exemplos com pytest e mencionamos unittest), cobrindo casos de uso típicos e extremos – desde entradas válidas comuns até situações de erro e fronteira. Exploramos boas práticas de escrita de testes (nomes claros, isolamento, cobertura de casos de borda, uso adequado de mocks etc.) para produzir uma suíte de testes eficaz e de fácil manutenção.

No hands on, vimos na prática a criação de alguns testes para uma função de exemplo e executamos esses testes, inclusive simulando um erro e sua detecção. Também explicamos como configurar uma pipeline de Integração Contínua (CI) simples (com GitHub Actions) para rodar os testes automaticamente a cada push no repositório, assegurando qualidade contínua. Ou seja, consolidamos a ideia de automatizar a verificação: o próprio sistema avisa se algo se quebrar.

Aprofundamos o funcionamento e os benefícios da Integração Contínua: entendemos a definição formal (merge frequente + build/test automático) e como isso se traduz em práticas do dia a dia (pipelines de CI monitorando cada commit). Vimos comparativamente as principais ferramentas de CI do mercado (Jenkins, GitHub Actions, GitLab CI), analisando seus pontos fortes e fracos para entender como escolher a melhor para cada cenário.

Em suma, você aprendeu que testes automatizados e CI andam juntos para manter a qualidade de bibliotecas internas: os testes fornecem a cobertura e confiança e a integração contínua os coloca em ação a cada mudança, garantindo que a biblioteca continue sólida mesmo com múltiplos(as) desenvolvedores(as) contribuindo ao longo do tempo.

Agora você está apto(a) a aplicar essas práticas nos seus projetos – e lembre-se, esse conhecimento é não só teoricamente valioso, mas altamente requisitado no

mercado de trabalho moderno de software. Continue praticando, escrevendo novos testes e refinando pipelines, pois quanto mais você exercitar, mais natural ficará desenvolver com qualidade contínua em mente!

EMENDAS

REFERÊNCIAS

CONTINUOUS DELIVERY FOUNDATION (CDF). **Relatório State of CI/CD 2024**. 2024. Disponível em: <https://www.devopsdigest.com/state-of-cicd-2024>. Acesso em: 21 jan. 2026.

FOWLER, M. **Continuous Integration**. 2024. Disponível em: <https://martinfowler.com/articles/continuousIntegration.html>. Acesso em: 21 jan. 2026.

GEEKSFORGEES. **Difference between Pytest and Unittest**. 2025. Disponível em: <https://www.geeksforgeeks.org/software-testing/difference-between-pytest-and-unittest/>. Acesso em: 21 jan. 2026.

MAPREE. **A Importância dos Testes Automatizados**. 2024. Disponível em: <https://www.mapree.dev/blog/testes-automatizados-a-importancia>. Acesso em: 21 jan. 2026.

MICCO, J. **The State of Continuous Integration Testing @ Google**. 2013. Disponível em: <https://research.google.com/pubs/archive/45880.pdf>. Acesso em: 21 jan. 2026.

SOUZA, A. L. **Gerando Testes Automatizados com IA para Códigos Mais Complexos**. 2025. Disponível em: <https://dev.to/asouza/gerando-testes-automatizados-com-ia-para-codigos-mais-complexos-42fh>. Acesso em: 21 jan. 2026.

PALAVRAS-CHAVE

Testes automatizados. Integração contínua. Qualidade de software.

EMENDAS



POSTECH